מסמך אפיון מעודכן לבניית הפרויקט

מערכת להזמנת חדרים בבתי מלון

ASP.NET Core Web API + SQL Server + React

מותאם לדרישות פרויקט הסיום של קורס .NET

גרסה ממוקדת: ללא WeatherService וללא API חיצוני שאינם דרישות חובה

מטרת המסמך

המסמך מגדיר את מבנה מערכת המלונות ואת סדר המימוש, תוך התאמה לדרישות המורה. הדגש הוא על שרת Web API תקין, ארכיטקטורה נקייה, עבודה מלאה עם EF Core, אבטחה, בדיקות, וליבה עסקית שמטפלת נכון בתחרות על משאב מוגבל.

המשאב המוגבל בפרויקט

המשאב המוגבל הוא חדר מסוים במלון עבור טווח תאריכים מסוים. שתי בקשות יכולות לנסות להזמין את אותו חדר כמעט בו-זמנית, אך אסור ששתיהן יצליחו. המערכת תשתמש ב-Optimistic Concurrency ותדחה את הבקשה המתנגשת עם HTTP 409 Conflict.

1. מבנה ה-Solution

```
HotelProject.sln
|-- Hotel.Core
|   |-- Entities
|   |-- DTOs
|   |-- Interfaces
|   `-- Enums
|-- Hotel.Data
|   |-- Context
|   |-- Configurations
|   |-- Repositories
|   `-- Migrations
|-- Hotel.Service
|   |-- Services
|   `-- Mapping
`-- Hotel.API
    |-- Controllers
    |-- Middleware
    |-- Extensions
    `-- Program.cs
```

כיוון התלויות

- Hotel.Core אינו מפנה לפרויקט אחר ואינו תלוי ב-EF Core או בחבילות תשתית.
- Hotel.Data מפנה ל-Hotel.Core.

- Hotel.Service מפנה ל-Hotel.Core.
- Hotel.API מפנה ל-Hotel.Service ול-Hotel.Core, ומכיר את Hotel.Data לצורך רישום התלויות ב-Program.cs.

2. Entities

ישות	מאפיינים מרכזיים	קשרים
Hotel	Id, Name, City, Address	Rooms (1:N), FavoritedByUsers (N:M)
Room	Id, RoomNumber, Floor, Price, HotelId, LastBookingAt, RowVersion	Hotel (N:1), Payments (1:N)
User	Id, Name, Email, PasswordHash, Role	Payments (1:N), FavoriteHotels (N:M)
Payment	Id, UserId, RoomId, CheckIn, CheckOut, Amount, IsPaid	User (N:1), Room (N:1)

Room.RowVersion הוא ה-Concurrency Token. ב-SQL Server הוא מוגדר כ-rowversion באמצעות [Timestamp] או Fluent API. בעת הזמנה מוצלחת Room.LastBookingAt מתעדכן כדי לגרום לעדכון שורת החדר ולשינוי ה-RowVersion.

3. קשרי גומלין ו-Fluent API

- Hotel 1:N Room.
- User 1:N Payment.
- Room 1:N Payment.
- User N:M Hotel דרך FavoriteHotels / FavoritedByUsers.
- Unique Index על User.Email.
- Index על Hotel.City.
- Payment.Amount ו-Room.Price מסוג decimal(10,2).
- Room.RowVersion מוגדר כ-Concurrency Token.
- הגדרות הקשרים והאינדקסים יבוצעו ב-Fluent API בתוך Configurations.

4. DTOs ו-Validation

Entities אינם נכנסים או יוצאים ישירות דרך ה-API. יהיו DTOs נפרדים ל-request ול-response, עם Data Annotations מתאימים. בדיקות עסקיות התלויות במצב בסיס הנתונים יבוצעו בשכבת Service.

תחום	DTO	שדות עיקריים
Auth	RegisterDto	Name, Email, Password
Auth	LoginDto	Email, Password
Auth	AuthResponseDto	Token, Name, Role
Hotels	CreateHotelDto	Name, City, Address
Hotels	UpdateHotelDto	Name, City, Address
Hotels	HotelResponseDto	Id, Name, City, Address
Rooms	CreateRoomDto	RoomNumber, Floor, Price, HotelId
Rooms	UpdateRoomDto	RoomNumber, Floor, Price
Rooms	RoomResponseDto	Id, RoomNumber, Floor, Price, HotelId
Rooms	AvailableRoomsQueryDto	CheckIn, CheckOut, Page, PageSize
Payments	CreatePaymentDto	RoomId, CheckIn, CheckOut
Payments	PaymentResponseDto	Id, RoomId, CheckIn, CheckOut, Amount, IsPaid

- [Required], [EmailAddress], [MinLength], [Range] לפי הצורך.
- Model Binding מתאים: [FromRoute], [FromQuery], [FromBody].
- ModelState / ApiController validation עבור קלט לא תקין.
- AutoMapper Profile ימוקם ב-Hotel.Service/Mapping כדי לשמור את Core נקי מתלות ב-AutoMapper.

5. Interfaces ו-Dependency Injection

Interfaces מוגדרים ב-Core. המימושים של repositories נמצאים ב-Data והמימושים של services נמצאים ב-Service. כל התלויות מוזרקות דרך constructor ואין יצירה ידנית של Service בתוך Service.

סוג	**Interfaces**
Services	IHotelService, IRoomService, IAuthService, IPaymentService
Repositories	IHotelRepository, וכן IGenericRepository<T> לפי הצורך העסקי IRoomRepository, IUserRepository, IPaymentRepository

DbContext ו-repositories הם Scoped. הרישום מתבצע ב-Program.cs.

6. EF Core, DbContext, Migrations ו-Seed

- AppDbContext עם Code First ו-DbSet עבור Hotels, Rooms, Users, Payments.
- SQL Server הוא בסיס הנתונים שנבחר לפרויקט.
- Connection String נטען מ-Configuration / User Secrets ואינו נכתב בקוד ואינו נשמר ב-repository.
- לפחות שתי Migrations בהיסטוריה.
- Migration ראשונה: יצירת הסכמה הראשונית.
- Migration שנייה: שינוי/Seed מתועד בהתאם למימוש, כך שבפועל קיימות לפחות שתי migrations.
- Seed Data שמאפשר להפעיל את המערכת מיד: מלונות, חדרים, Admin, Customers, הזמנות לדוגמה וקשרי FavoriteHotels.

הערה: קובצי migration בעלי timestamp בשם הם תקניים ונוצרים אוטומטית על ידי EF Core.

7. Repositories ו**שאילתות** EF Core

- CRUD מלא.
- קריאות Read-only עם AsNoTracking.
- חיפוש מלונות לפי City עם LINQ וסינון.
- חדרים פנויים לפי מלון וטווח תאריכים.
- Pagination אמיתי ב-SQL באמצעות Skip ו-Take לפני ToListAsync.
- Include / ThenInclude לפי הצורך וללא N+1.
- בדיקת חפיפה: existing.CheckIn < requestedCheckOut && existing.CheckOut > requestedCheckIn.
- כל פעולות ה-I/O אסינכרוניות ומקבלות CancellationToken.

8. AutoMapper

MappingProfile נמצא ב-Hotel.Service/Mapping. הוא ממפה בין DTOs לבין Entities בהתאם לכיוון הפעולה. לדוגמה: Hotel -> HotelResponseDto, CreateHotelDto -> Hotel, Room -> RoomResponseDto, CreateRoomDto -> Room, Payment -> PaymentResponseDto. המיפוי נרשם ב-Program.cs באמצעות Dependency Injection.

9. **שכבת** Service - **הלוגיקה העסקית**

Service	אחריות
HotelService	CRUD, חיפוש לפי עיר, מועדפים

RoomService	CRUD, חדרים פנויים לפי מלון ותאריכים, Pagination
AuthService	Register, Login, Password Hash, יצירת JWT
PaymentService	Validation, בדיקת זמינות, חישוב Amount, יצירת הזמנה, טיפול ב-Concurrency

אין WeatherService ואין External API. הם הוסרו מהאפיון משום שאינם דרישת חובה של המורה ומוסיפים מורכבות שאינה תורמת לציון.

10. זרימת הזמנה / Payment

- בדיקת CheckIn ו-CheckOut.
- טעינת Room במצב Tracking.
- בדיקה שאין Payment חופף לאותו Room.
- חישוב nights = (CheckOut - CheckIn).Days.
- חישוב Amount = nights * Room.Price בצד השרת.
- עדכון Room.LastBookingAt = DateTime.UtcNow.
- יצירת Payment.
- SaveChangesAsync באותה יחידת עבודה / טרנזקציה הנדרשת.
- תפיסת DbUpdateConcurrencyException והמרה ל-409 Conflict עם הודעה ברורה.

11. Controllers ו-REST

Controller	Endpoints מרכזיים
AuthController	POST /api/auth/register; POST /api/auth/login
HotelsController	GET /api/hotels?city=...; GET /api/hotels/{id}; POST; PUT; DELETE; favorite
RoomsController	GET /api/hotels/{hotelId}/rooms/available; GET /api/rooms/{id}; POST; PUT; DELETE
PaymentsController	POST /api/payments; GET /api/payments/my

- Controllers מזריקים Services בלבד - לא DbContext.
- Status codes תקינים: 200, 201 עם Location, 204, 400, 401, 403, 404, 409.
- שמות משאבים ברבים ופעולות באמצעות HTTP methods.

12. JWT Authentication & Authorization

- Register יוצר User עם PasswordHash ו-Role ברירת מחדל Customer.
- Login מאמת סיסמה ומחזיר JWT.
- Claims: UserId, Email, Role.
- Roles: Customer ו-Admin.
- [Authorize] לפעולות משתמש מחובר.
- [Authorize(Roles = "Admin")] לפעולות ניהול מתאימות.
- JWT Key נשמר ב-User Secrets ולא ב-repository.

13. Optimistic Concurrency - דרישת הליבה

המערכת חייבת להוכיח ששתי בקשות המתחרות על אותו חדר אינן יכולות שתיהן להצליח. Room.RowVersion הוא ה-concurrency token. כאשר שתי בקשות קראו את אותה גרסה, הראשונה שומרת בהצלחה והשנייה מקבלת DbUpdateConcurrencyException. ה-Service מתרגם את ההתנגשות ל-409 Conflict.

- Concurrency token מוגדר בקוד ובDB.
- טיפול ספציפי ב-DbUpdateConcurrencyException.
- בדיקת זמינות והשמירה מתבצעות באותה יחידת עבודה המתאימה.
- Unit Test עם שני DbContext נפרדים: הראשון מצליח והשני נכשל.
- README מסביר מהו המשאב המוגבל ואיך מטופלת התחרות.

14. Async ו-CancellationToken

כל שרשרת הקריאה אסינכרונית: Controller -> Service -> Repository -> EF Core. נעשה שימוש ב-ToListAsync, FirstOrDefaultAsync, SaveChangesAsync וכדומה, ו-CancellationToken מועבר לאורך כל השרשרת.

15. Middleware

- GlobalExceptionMiddleware - טיפול גלובלי בחריגות והחזרת JSON אחיד.
- CorrelationIdMiddleware - יצירת מזהה לכל בקשה והכנסתו ללוגים.
- סדר pipeline: Exception Middleware -> CorrelationId -> Authentication -> Authorization -> MapControllers.

16. NLog

- NLog.Web.AspNetCore.
- nlog.config בפרויקט API.
- Information לבקשות ולזרימה רגילה.
- Warning להתנגשות / 409.
- Error לחריגות.
- אין לכתוב ללוג סיסמאות, JWT tokens או מידע סודי.
- הקוד משתמש ב-ILogger הרגיל של NET; NLog. מחובר כתשתית logging.

17. Configuration ו-Secrets

תוכן	מיקום
הגדרות לא רגישות	appsettings.json
הגדרות פיתוח לא רגישות	appsettings.Development.json
Connection String, JWT Key	User Secrets

אין External API Key משום שה-Weather API הוסר מהפרויקט.

18. Swagger

- Swagger פעיל בסביבת Development ומתעד את כל ה-endpoints.
- Bearer/JWT מוגדר ב-Swagger לצורך בדיקת endpoints מוגנים.
- כל endpoints נבדקים ב-Swagger לפני בניית React.

Unit Tests .19

Test	מטרה
HotelService_GetByCity	בדיקת חיפוש לפי עיר
RoomService_AvailableRooms	בדיקת זמינות חדרים
PaymentService_InvalidDates	דחיית תאריכים לא תקינים
PaymentService_CalculatesAmount	חישוב סכום בצד השרת
PaymentService_RoomAlreadyBooked	דחיית חפיפה
Concurrency_TwoDbContexts	חובה: הוכחת optimistic concurrency

Service tests משתמשים ב-xUnit וב-Moq מול repository interfaces. בדיקת ה-concurrency יכולה להשתמש במסד נתונים מתאים שמסוגל להוכיח את מנגנון ה-concurrency בפועל.

מינימום נדרש - React .20

הלקוח נועד להדגים את השרת בלבד, ולכן אין צורך להשקיע בעיצוב מעבר לנדרש.

מסך	תוכן
Login/Register	שמירת JWT ושליחתו ב-header
מלונות	בחירת עיר וטעינת מלונות
חדרים	תאריכים, חדרים פנויים ו-Pagination
הזמנה	לשרת POST
409	הודעה ברורה שהחדר נתפס / אינו זמין

בדיקות סופיות לפני הגשה .21

- HotelProjectDb קיים ונוצר באמצעות migrations.
- Hotels, Rooms, Users, Payments וטבלת הקישור Many-to-Many קיימות.
- Seed Data קיים.
- Foreign Keys תקינים.
- User.Email ייחודי.
- Room.RowVersion קיים ומתעדכן.
- לפחות שתי Migrations קיימות.
- POST /api/payments מוסיף הזמנה בפועל.
- חדר עם הזמנה חופפת אינו מופיע ברשימת החדרים הפנויים.
- Build של כל ה-Solution עובר ללא שגיאות.
- כל endpoints נבדקו ב-Swagger.

README-ו GitHub .22

- שני repositories נפרדים: Server ו-Client.
- gitignore. תקין ללא bin, obj, node_modules או secrets.
- README בשרת: תיאור המערכת, המשאב המוגבל, Optimistic Concurrency, הוראות הרצה, בסיס הנתונים הנדרש, פקודות migrations ומשתמשי Demo לכל Role.
- המערכת עולה ורצה מקומית ללא התערבות ידנית בבסיס הנתונים.

23. סדר ביצוע מעודכן וממוקד

הסדר הבא משקף את מה שכבר נבנה ואת מה שנשאר, בלי לפצל לעשרות שלבים קטנים:

שלב	תכולה
1 - בסיס ונתונים	Solution, Entities, DTOs, DbContext, Configurations, Migrations, Repository - כבר בוצע ברובו
2 - Mapping + Services	AutoMapper, Service interfaces/implementations, לוגיקה עסקית, Async + CancellationToken
3 - API + Security	Controllers, REST, Validation, DI, JWT, Roles, Swagger auth
4 - הזמנות ותחרות	Available Rooms, Pagination, Payment flow, Optimistic Concurrency, 409
5 - תשתיות ואיכות	Middleware, CorrelationId, NLog, Configuration/User Secrets, Seed + migration נוספת
6 - בדיקות וסיום	Unit Tests, 2 DbContext test, Swagger/SQL verification, README/GitHub, React מינימלי

24. דברים שאינם נדרשים בפרוויקט

- WeatherService / Weather API - הוסר.
- External API אחר - לא נדרש.
- Cloud deployment - לא נדרש.
- Docker - לא נדרש.
- S3 / file storage - לא נדרש.
- Google Login - לא נדרש.
- Caching, SignalR, CI - לא נדרשים.

עיקרון מנחה

הפרויקט נשאר קטן ומדויק: ארבע ישויות מרכזיות, שכבות נקיות, **DTOs, Services, REST**, אבטחה, **EF Core**, בדיקות וטיפול נכון בתחרות על חדר. המטרה היא לעמוד בכל דרישות החובה בצורה מלאה ולא להוסיף רוחב שאינו נדרש.